MLOps Assignment 1: Heart Disease Risk Prediction

Course: Machine Learning Operations (AIMLCZG523), Assignment 01

Student name	ARWADE RUTUJA NITIN
BITS ID	2025ae05375
Email	2025ae05375@wilp.bits-pilani.ac.in
Code repository	https://github.com/Rutuja-arwade/mlops-heart-disease
Pipeline video	https://drive.google.com/file/d/1BIK8L5S573MXqQtXSKFjAlnojAxp2jKt/view?usp=sharing

Contents

1. Introduction 1
2. Setup and installation 1
3. Data acquisition and EDA 2
4. Feature engineering and model development 3
5. Experiment tracking (MLflow) 5
6. Architecture diagram 6
7. CI/CD pipeline and automated testing 6
8. Model containerization 7
9. Production deployment (Kubernetes) 8
10. Monitoring and logging 9
11. Repository and deliverables 10

1. Introduction

The task is to predict the presence of heart disease from 13 clinical attributes of the UCI Heart Disease dataset (Cleveland subset, 303 patients). The model is served as a FastAPI service in a Docker container, deployed on Kubernetes (Minikube) and monitored with Prometheus and Grafana. The whole pipeline (lint, tests, training, Docker build, deployment test) runs on GitHub Actions.

The final model is a logistic regression with mean cross-validated ROC-AUC 0.907 and, on the held-out test set, ROC-AUC 0.961, accuracy 0.869 and recall 0.929.

2. Setup and installation

Requirements: Python 3.11 and Docker. Minikube, kubectl and Helm are needed only for the Kubernetes part.

```
git clone https://github.com/Rutuja-arwade/mlops-heart-disease.git
cd mlops-heart-disease
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt

python -m src.data      # download and clean the data
python -m src.eda       # EDA figures
python -m src.train     # train, track with MLflow, export model to models/
pytest                  # unit tests
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5001
```

Docker:

```
docker build -t heart-disease-api:1.0.0 .
docker run -d -p 8000:8000 heart-disease-api:1.0.0
curl -X POST localhost:8000/predict -H "Content-Type: application/json" \
     -d @sample_request.json
```

Kubernetes (Minikube) and monitoring:

```
minikube start --driver=docker
minikube addons enable ingress
minikube addons enable metrics-server
minikube image load heart-disease-api:1.0.0
helm upgrade --install heart deploy/helm/heart-disease-api \
     -n heart-disease --create-namespace --wait
kubectl apply -k monitoring
minikube tunnel
```

Access to the deployed API. The API is not hosted publicly. After the steps above it is available at http://127.0.0.1:8080 (Swagger UI at /docs) through the LoadBalancer service, and through the NGINX Ingress with host name heart-api.local.

3. Data acquisition and EDA

Data. The file processed.cleveland.data from the UCI repository is downloaded by `src/data.py` (also `scripts/download_data.sh`). The script retries, falls back to a second UCI mirror and checks a SHA-256 checksum. The cleaned file is `data/processed/heart_disease_clean.csv`.

Cleaning. "?" is read as missing, values outside plausible medical ranges or with unknown category codes are set to missing, the diagnosis `num` (0 to 4) is converted to a binary target (1 if num > 0), and duplicates are dropped. The result has 303 rows. Only `ca` (4 rows) and `thal` (2 rows) have missing values. They are not filled during cleaning but inside the model pipeline (median or most frequent value), so the imputer is fitted on training data only.

EDA. Eight figures are in `reports/figures` and in the notebook `notebooks/01_eda.ipynb`. The main observations:

- The classes are nearly balanced: 164 patients without disease (54 %) and 139 with disease (46 %).
- The features most correlated with the target are `thal` (0.53), `ca` (0.46), `exang` (0.43), `oldpeak` (0.42), `thalach` (-0.42) and `cp` (0.41). No pair of features is strongly collinear (|r| < 0.6).
- Patients with disease are older, reach a lower maximum heart rate and have more ST depression. Cholesterol and resting blood pressure overlap heavily between the classes.
- `cp`, `restecg`, `slope` and `thal` are category codes (for example chest pain type 4 has a 73 % disease rate, the other types 18 to 30 %), so they are one-hot encoded.

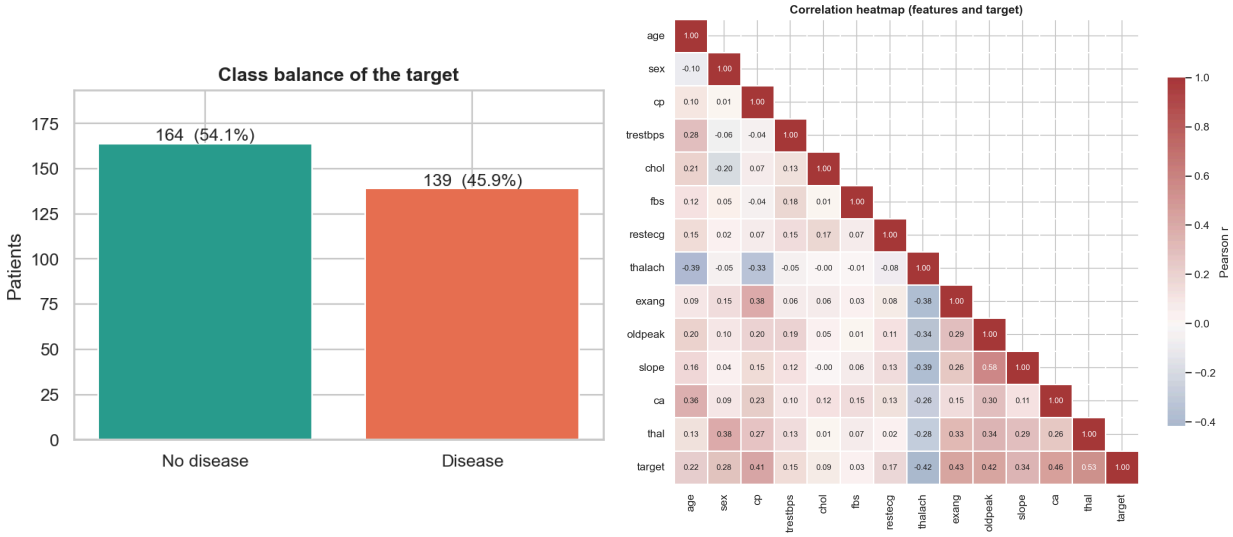


Figure 1: class balance (left) and correlation heatmap (right).

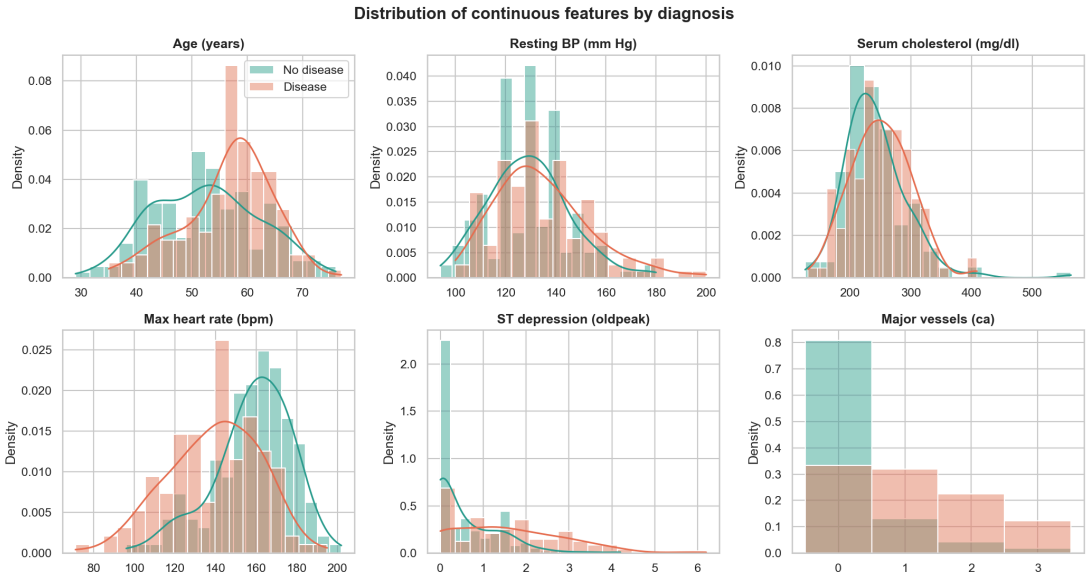


Figure 2: distribution of the continuous features by diagnosis.

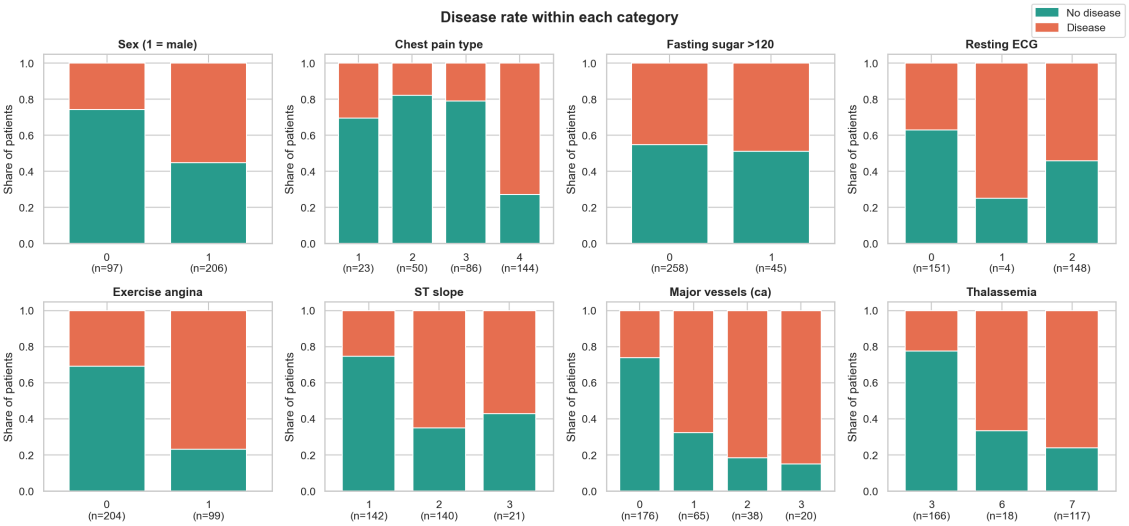


Figure 3: disease rate within each category of the nominal and binary features.

4. Feature engineering and model development

Pipeline. One scikit-learn Pipeline (`src/features.py`) is used for training, cross-validation and serving:

Step	Columns	Transformation
Feature builder	age, thalach, trestbps	adds hr_achieved_ratio = thalach / (220 - age) and rate_pressure_index = thalach x trestbps / 1000 (can be switched off)
Continuous	age, trestbps, chol, thalach, oldpeak, ca	median imputation, standard scaling
Nominal	cp, restecg, slope, thal	most frequent imputation, one-hot encoding
Binary	sex, fbs, exang	most frequent imputation

Training and tuning. The data is split 80/20 (stratified, 242 training and 61 test patients, random_state 42). Each model is tuned with stratified 5-fold cross-validation on the training part and the metrics accuracy, precision, recall, F1 and ROC-AUC. The best setting is chosen by mean CV ROC-AUC. The test set is used only for the final result.

Model	Search	Candidates
Majority-class baseline	none	1
Logistic Regression	grid over C, L1/L2, class weight, engineered features on/off	56

Model	Search	Candidates
Random Forest	random search over trees, depth, leaf size, max features, class weight	30
Gradient Boosting	random search over trees, learning rate, depth, subsample, leaf size	25

Results.

Model	CV ROC-AUC	Test accuracy	Test precision	Test recall	Test ROC-AUC
Logistic Regression	0.907 +/- 0.019	0.869	0.812	0.929	0.961
Random Forest	0.905 +/- 0.023	0.885	0.839	0.929	0.955
Gradient Boosting	0.892 +/- 0.029	0.885	0.862	0.893	0.938
Baseline	0.500	0.541	0.000	0.000	0.500

Model selection. Logistic regression (L2 penalty, C = 0.3, balanced class weights) has the best CV ROC-AUC and the lowest variance between folds. Random forest is almost equal, but with equal performance the simpler and easier to explain model was chosen. On the test set the model misses 2 of 28 patients with disease (27 true negatives, 6 false positives, 2 false negatives, 26 true positives). The engineered features did not help logistic regression, so the final model does not use them.

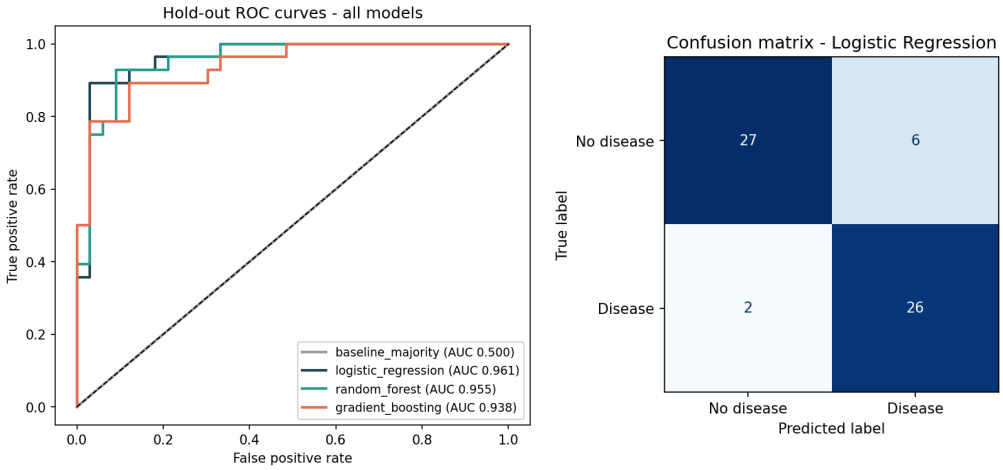


Figure 4: ROC curves of all models on the test set (left) and confusion matrix of the final model (right).

The coefficients of the final model are medically sensible. Risk goes up with more major vessels (ca), male sex, asymptomatic chest pain, a reversible thallium defect (thal = 7) and exercise-induced angina. Risk goes down with a normal thallium scan (thal = 3), an up-sloping ST segment and a higher maximum heart rate.

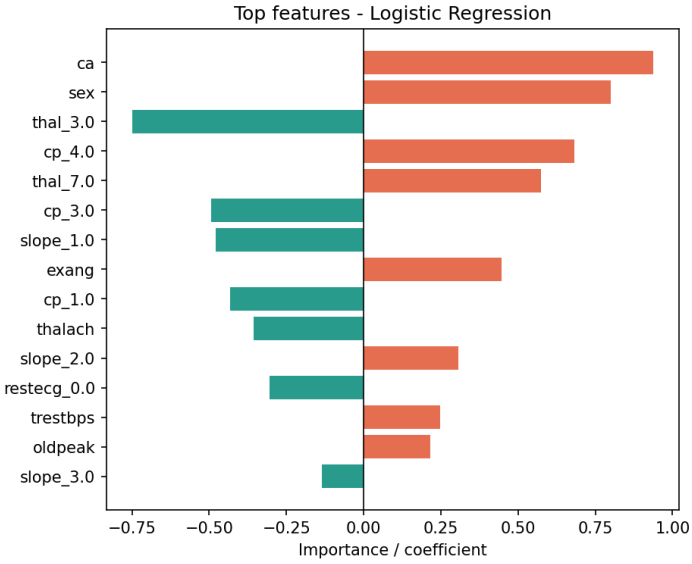


Figure 5: largest coefficients of the final logistic regression.

5. Experiment tracking (MLflow)

MLflow is used with a SQLite backend (`mlflow.db`, needed for the model registry) and file artifacts (`mlruns/`), experiment name `heart-disease-risk`.

- One parent run per model family, and one nested run for each of the 112 hyperparameter trials (parameters, mean and standard deviation of the 5 CV metrics).
- Parent runs log the best parameters, CV and test metrics, plots (ROC, precision-recall, confusion matrix, calibration, feature importance), the CV results table and the fitted pipeline with signature.
- A `model-selection` run stores the comparison of all models.
- Every run has tags: git commit, dataset checksum and where it was run (local or github-actions).
- The final model is registered as `heart-disease-classifier` (version 1, alias `champion`) and exported to `models/model.joblib` (full pipeline) and `models/mlflow_model/`, together with `models/model_metadata.json`.

Reproducibility: all random steps use seed 42, versions are pinned in `requirements.txt`, and training again in a fresh copy of the repository produced a model file with the same SHA-256.

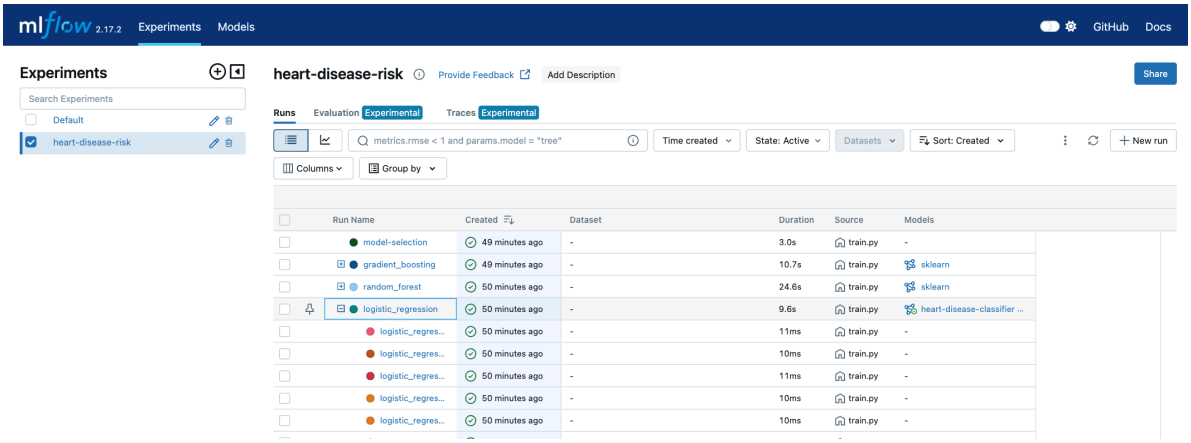


Figure 6: MLflow experiment with parent and nested trial runs.



Figure 7: model registry with the champion alias.

6. Architecture diagram

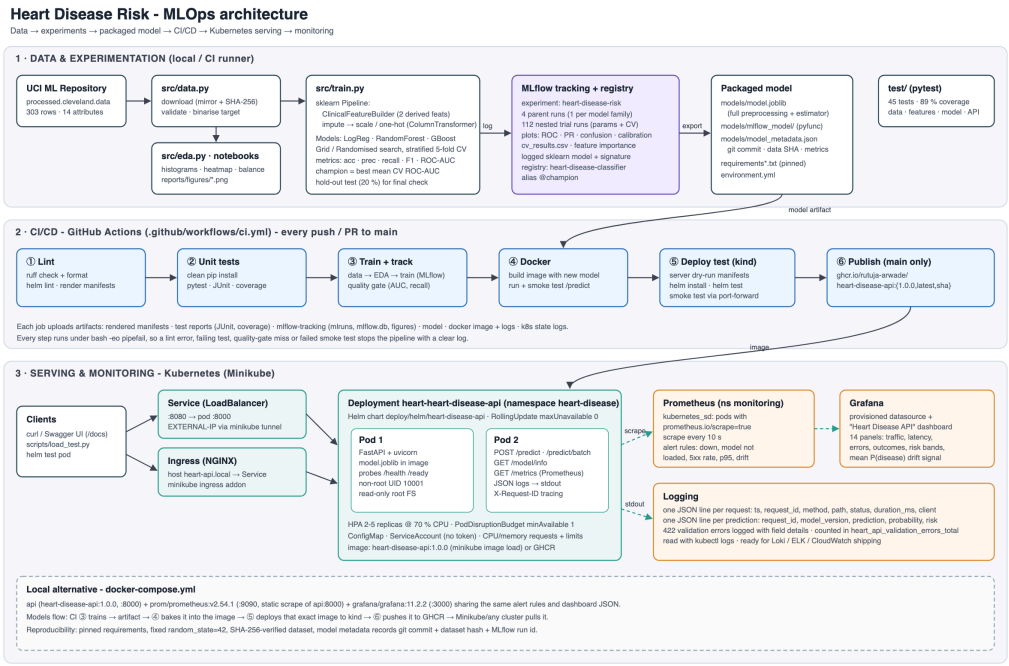


Figure 8: architecture of the pipeline.

7. CI/CD pipeline and automated testing

Unit tests. `test/` contains 11 pytest tests: data cleaning (test_data.py), feature engineering, model and prediction code (test_model.py), and the API endpoints (test_api.py). All 11 tests pass:



Figure 9: pytest run with coverage.

GitHub Actions (`.github/workflows/ci.yml`) runs on every push and pull request to main. A job runs only if the previous one passed:

Job	Steps	Artifacts
Lint	ruff, helm lint, render manifests	rendered manifests
Unit tests	clean pip install, download data, pytest with coverage	test reports
Train	data, EDA, training with MLflow, quality gate (fails if CV or test ROC-AUC < 0.85 or test recall < 0.75)	model, MLflow store, logs
Docker	build image with the new model, run container, smoke test	image, logs
Deploy (kind)	temporary Kubernetes cluster, Helm install, helm test, smoke test	cluster logs
Publish	push image to GitHub Container Registry (main branch only)	-

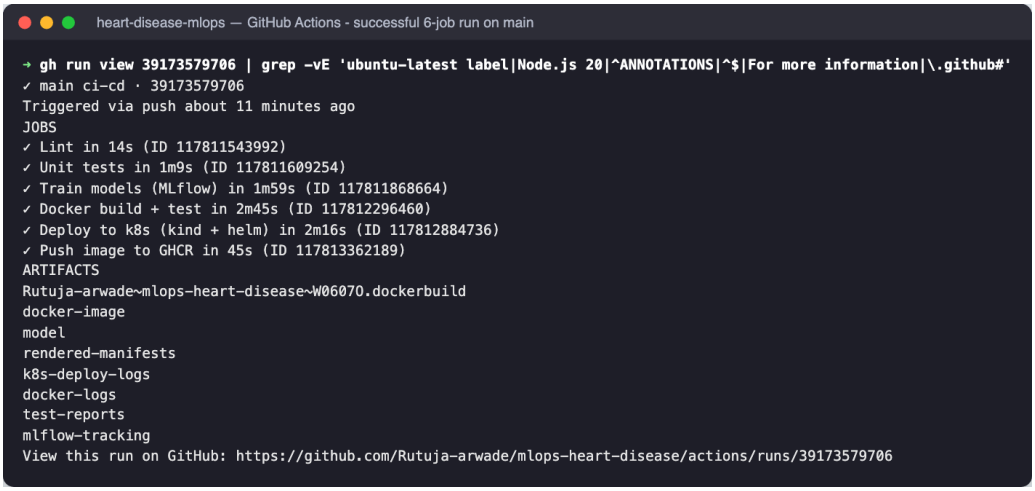


Figure 10: successful workflow run on main (all 6 jobs passed).

The training job prints the result of every model and the champion, and the quality gate then checks the thresholds:

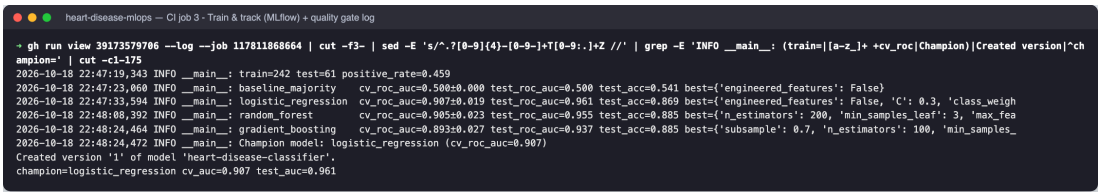


Figure 11: log of the training job in GitHub Actions.

Failure behaviour. Pull request #1 contained a deliberate bug (the threshold comparison in `src/predict.py` was reversed). The unit test job failed with a clear message (2 failed, 9 passed), and the train, Docker, deploy and publish jobs were skipped. The pull request was closed without merging.

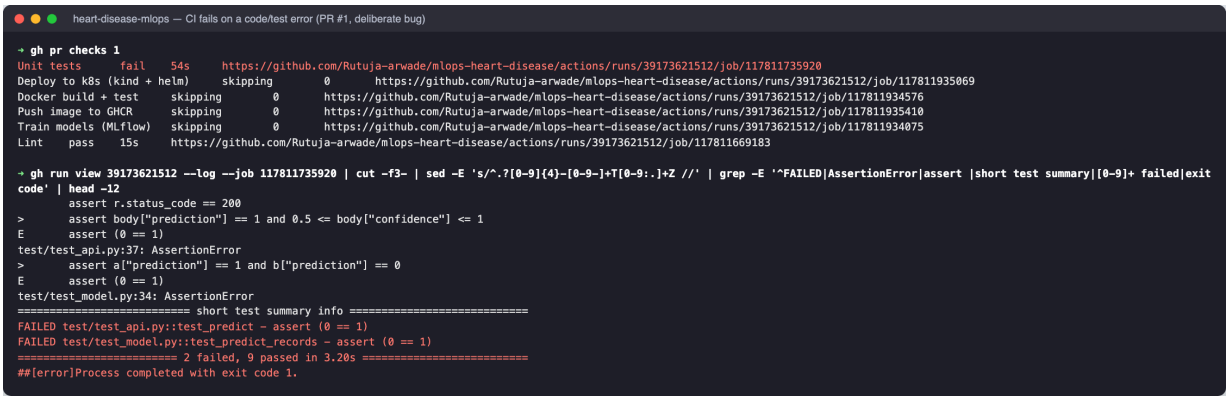


Figure 12: pipeline failure on a test error (pull request #1).

8. Model containerization

The API (`src/api/app.py`, FastAPI) is packaged with a multi-stage `Dockerfile` (python:3.11-slim, image size 169 MB, runs as non-root user, with a health check). Main endpoints:

Endpoint	Purpose
POST /predict	JSON with the 13 attributes (ca and thal optional). Returns prediction (0 or 1), confidence, probability of disease, risk level, model version and request id
POST /predict/batch	up to 500 records
GET /health, GET /ready	liveness and readiness
GET /metrics	Prometheus metrics

Inputs are validated with pydantic (allowed values and ranges, unknown fields rejected). Invalid requests get HTTP 422. Example:

```
Request (POST /predict):
{"age":63,"sex":1,"cp":4,"trestbps":150,"chol":286,"fbs":0,"restecg":2,
 "thalach":108,"exang":1,"oldpeak":1.5,"slope":2,"ca":3,"thal":7}

Response (200 OK):
{"prediction":1,"label":"heart_disease","confidence":0.9978,
 "probability_disease":0.9978,"risk_level":"high",
 "model_version":"1.0.0+logistic_regression", ...}
```



Figure 13: container run and smoke test with the sample input.

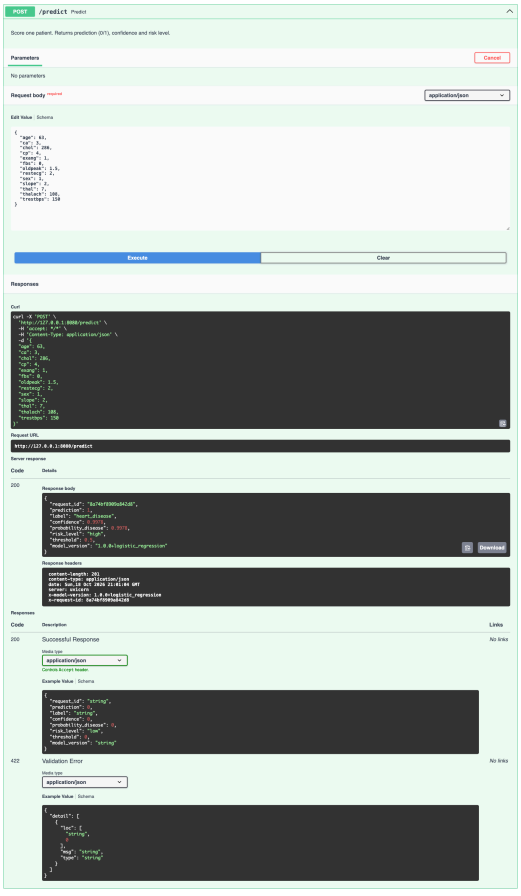


Figure 14: /predict called from the Swagger UI of the deployed service.

9. Production deployment (Kubernetes)

The container was deployed to Minikube (Kubernetes v1.31) with a Helm chart (`deploy/helm/heart-disease-api`). Plain manifests with the same objects are in `deploy/k8s`. The deployment has 2 replicas with rolling updates, health probes and resource limits, a LoadBalancer service (port 8080, made reachable by

`minikube tunnel`), an NGINX Ingress for the host heart-api.local, a horizontal pod autoscaler (2 to 5 pods) and a pod disruption budget. A `helm test` calls /predict from inside the cluster. Both the LoadBalancer and the Ingress were verified with real /predict requests.

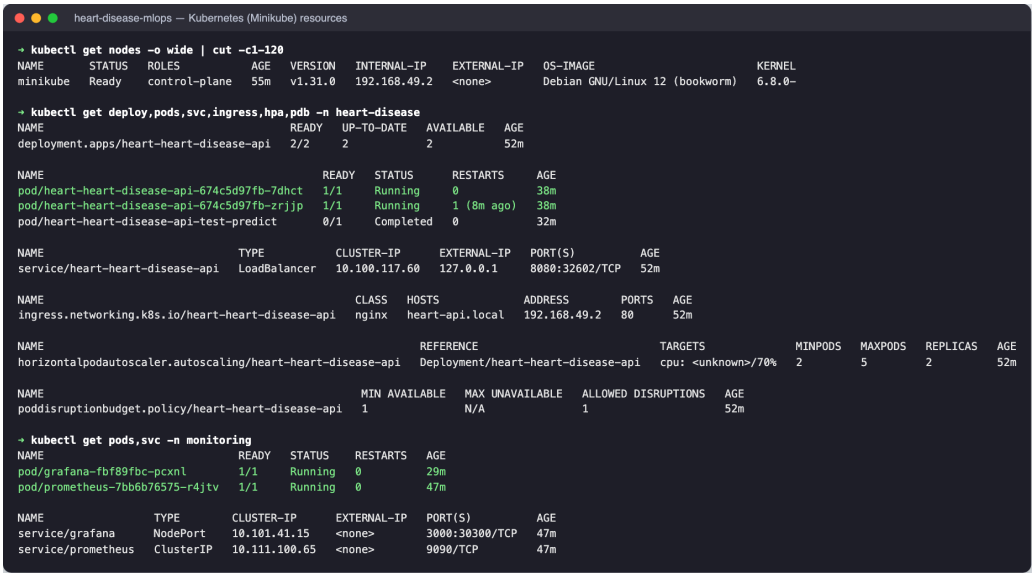


Figure 15: Kubernetes resources.

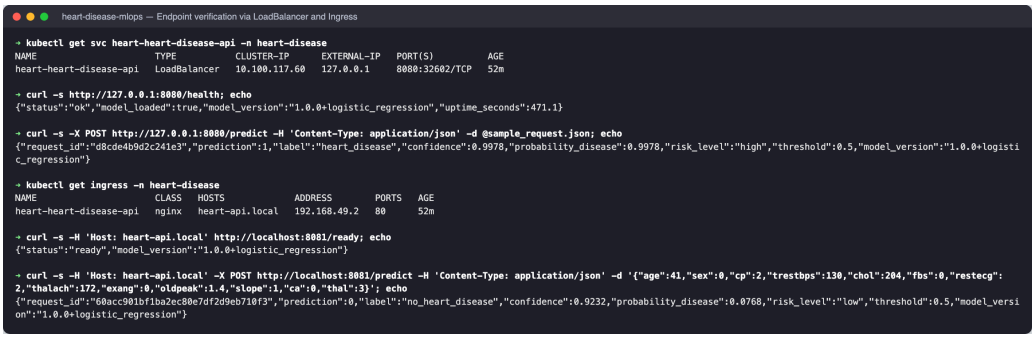


Figure 16: endpoint verification through the LoadBalancer and the Ingress.

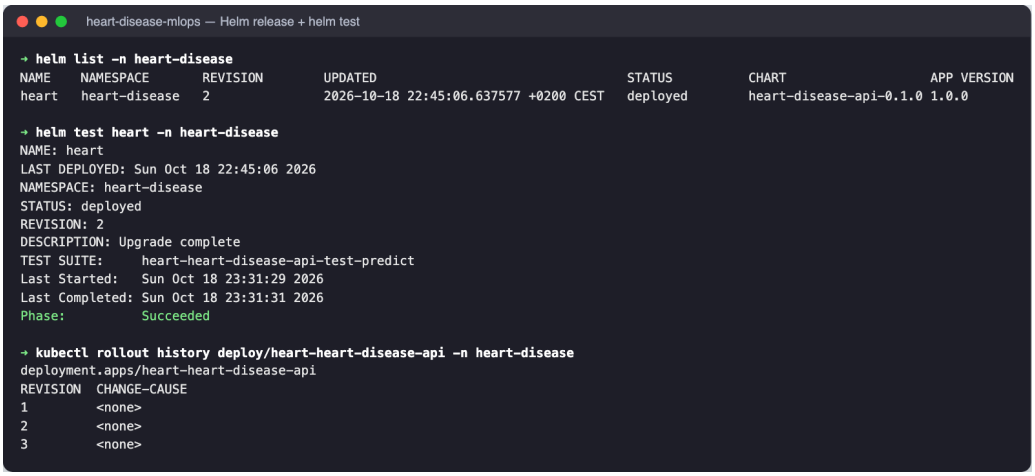


Figure 17: Helm release and the passing helm test.

10. Monitoring and logging

- **Logging.** Every request is logged as one JSON line (request id, path, status, duration), and every prediction as another line (model version, prediction, probability).
- **Metrics.** The API exposes Prometheus metrics at /metrics: request count and latency, predictions by outcome, predicted probability distribution, validation errors and model status.
- **Prometheus** finds the API pods in Kubernetes and scrapes them. Alert rules cover API down, model not loaded, high 5xx rate, slow responses and drift in predicted probability.

- **Grafana** shows a dashboard with 14 panels (traffic, latency, errors, predictions). A load test with 600 requests produced no server errors.



Figure 18: Grafana dashboard on Kubernetes during a load test.

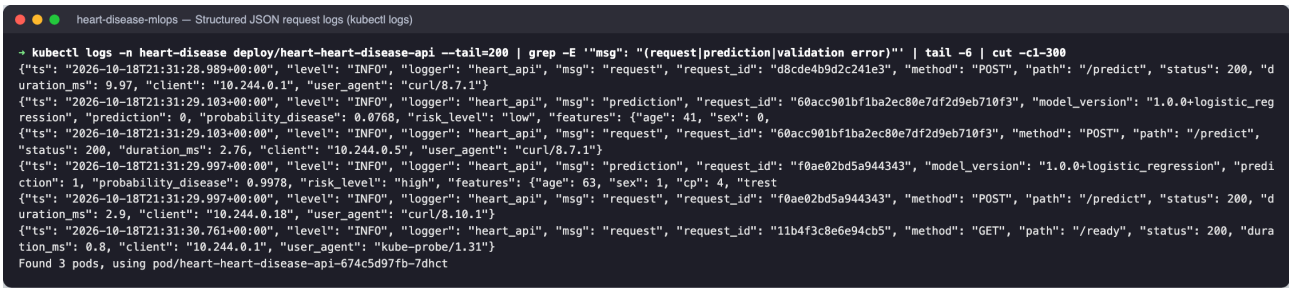


Figure 19: structured JSON logs from the API pods.

11. Repository and deliverables

All files are in the repository: https://github.com/Rutuja-arwade/mlops-heart-disease

Deliverable	Location
Code, Dockerfile, requirements and environment file	src/, Dockerfile, requirements*.txt, environment.yml
Cleaned dataset and download script	data/processed/heart_disease_clean.csv, scripts/download_data.sh
Notebooks (EDA, training, inference)	notebooks/
Unit tests	test/
GitHub Actions workflow	.github/workflows/ci.yml
Deployment manifests and Helm chart	deploy/, monitoring/
Screenshots	screenshots/
Report	report.pdf
Access instructions for the API	Section 2 of this report
Video of the pipeline	https://drive.google.com/file/d/1BIK8L5S573MXqQtXSKFjAlnojAxp2jKt/view?usp=sharing (a copy is in video/pipeline_walkthrough.mp4 in the repository)